

Graded Assignment on RAG

Name: Allen Michael Arendon

Date: October 7, 2026

Project: AI-Tee — IT helpdesk assistant

1. Public URL

<https://ai-tee.vercel.app>

2. Source Repository

<https://github.com/allenarendon/rag-chatbot-ai>

The README covers setup, seeding every PDF in `data/`, the chat UI, LangSmith environment variables, and the Vercel deploy steps.

3. Reflection

a. Corpus. I indexed twenty IT helpdesk work instructions, WI-01 through WI-20. They cover the questions a service desk actually gets: passwords, onboarding, devices, email, Wi-Fi, VPN, printers, access, security incidents, mobiles, meetings, backups, business apps, phones, room AV, offboarding, assets, patching, and browsers. One domain with numbered procedures is a better test of retrieval than unrelated articles, because the bot has to pick the right instruction and follow its steps, identity checks, and escalation rules.

`data/sources.json` gives each PDF a title, reference, summary, and topics so the model can choose before it searches.

b. Trickiest decision. Several of these PDFs have a broken cross-reference table, so `pdf-parse` returns almost no text even though the page content is still in compressed streams. Dropping those files would have left holes in the helpdesk. The seed script falls back to decoding those streams only when the normal parser returns too little text. That choice matters more than chunk size: a bad extract becomes a bad answer for every question that depends on that file.

c. What it does well. AI-Tee is strongest on a single, named procedure. A locked account, an offline printer, or a reported phishing email gets the next step first, the work-instruction title in the answer, and the retrieved passages under Sources with title, page, and match score. Questions outside IT helpdesk work stay short and do not search the index.

d. What it does badly. It is weakest when one sentence could belong to two instructions. "The app is slow," "email will not open on my phone," or "Wi-Fi is down so the VPN failed" can pull neighboring documents into the top four hits. The model then blends steps, or it sounds sure because the catalog summary is already in the prompt even when the matching passage is the wrong section. Flattened PDF text also makes some source cards harder to read than the original page.

e. Next week. I would chunk on the instruction headings instead of a fixed 800-character window, and filter retrieval by document reference so a password question cannot return a mobile-device chunk. I would keep a small set of graded questions, including the ambiguous ones, and read the LangSmith traces to see whether a miss came from the embedding, the top-k cutoff, or the final wording. A reranker on those four hits would come last, once I could measure whether it helped.

4. Optional Demo Questions

Questions that usually land on the right instruction:

1. I got locked out of my email after too many password attempts. What should Tier 1 try first, and when does this become a security issue?
2. The office printer says offline and it is the only printer for the team. Where do I start, and what priority is that?
3. A new hire starts Monday. What has to be approved before we create the account?
4. A user forwarded a suspicious email that asked them to sign in. What do we do in the first few minutes?
5. The meeting-room display is dead and a client meeting starts in 20 minutes. Who handles it, and how fast?

Questions that usually go wrong or stay thin:

6. The app is slow. Fix it.
7. Email will not open on my phone, and I also got a weird MFA prompt.
8. Wi-Fi dropped, so I think the VPN is broken. Walk me through both.
9. Someone leaves tomorrow. Revoke their access and collect the laptop in one answer.
10. How do I reset the password on my personal Gmail?

The first five map cleanly to WI-01, WI-08, WI-02, WI-10, and WI-16. The last five either span two instructions, are too vague for the top four hits, or invite the bot to apply a company procedure to an account the corpus does not cover.

5. Stretch Goals

a. LangSmith. Chat, embeddings, and seeding are traced in the `ai-tee` project. The AI SDK calls are wrapped so prompts and completions show up as runs, `getInformation` is traced as the retriever, and each chat request is one parent run named `ai-tee`. If LangSmith is unreachable, the handler logs the failure and still answers. The API key stays server-only.

b. Multi-PDF support and metadata. `npm run seed` reads every PDF directly in `data/`, chunks it, and stores vectors as `filename#index` with metadata for the chunk text, page, source filename, catalog title, and topics. Each embedding is prefixed with the document title and topics so similar steps in different instructions are easier to tell apart. The chat route returns that metadata as Sources, and the catalog in the system prompt lists which instruction to search. Reseeding clears the index first, so a removed PDF does not keep answering.